



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

M. Niemetz

Datentypen und Arrays

Kassenautomat

November 2025, Version: 1.35

Fakultät Elektro- und Informationstechnik

1 Einleitung

Diese Aufgabe beschäftigt sich neben der schon bekannten Aufteilung eines Programms in Funktionen und der Verwendung von Arrays mit einer Verwendung nutzerdefinierter Datenstrukturen.

Ziel der Aufgabe ist es, einen Teil eines Kassensautomaten zu implementieren, und zwar den Algorithmus zur Geldausgabe, bei der es darauf ankommt, den Betrag in **möglichst wenige** einzelne Scheine bzw. Münzen aufzuteilen.

2 Der Kassenautomat

Wichtig: Verwenden Sie in dieser Aufgabe den Datentyp **float** und **nicht double** zur Speicherung der Geldwerte¹.

2.1 Berechnung der Anzahl für eine gegebene Geldeinheit

Beginnen Sie Ihre Arbeit, indem Sie als ersten einfachen Teilschritt eine Funktion entwickeln, welche für eine Gesamtsumme G und einen einzelnen *festgelegten* Stückelungswert S die Anzahl an notwendigen Geldeinheiten $n(G, S) \in \mathbb{N}$ berechnet, um eine möglichst große Summe des Betrags G auszubezahlen und einen möglichst kleinen Rest zurückzubehalten.

Die Funktion $n(G, S)$ soll also folgende Eigenschaften besitzen:

$$n(G, S) \cdot S \leq G$$

und

$$(n(G, S) + 1) \cdot S > G$$

Überlegen Sie, wie Sie den Wert der Funktion $n(G, S)$ definieren bzw. berechnen können.

Implementieren Sie danach Ihre Lösungsidee als C-Funktion. Diese soll die Anzahl n als Rückgabewert der Funktion in Form eines **int**-Werts zurückliefern.

Hinweis: Testen Sie Ihre Funktion gründlich und dokumentieren Sie das Ergebnis!



Schreiben Sie eine Funktion, die die Anzahl an Geldeinheiten für eine bestimmte Schein- bzw. Münzart berechnet.

Dozentenfrage

Diskussion der gewählten Lösung

¹Für Experten: Die Verwendung von **float** für Geldsummen führt im weiteren Verlauf der Aufgabe zu Schwierigkeiten. Das ist beabsichtigt, damit Sie diese Probleme kennenlernen.

2.2 Implementierung des Kassen-Algorithmus

Nun soll das Projekt fortgesetzt werden, indem Sie die bereits entwickelte Funktion verwenden, um ein Hauptprogramm in der Funktion `main` zu schreiben, welches einen Algorithmus zur Berechnung der optimalen Stückelung umsetzt, d.h. den Betrag aus **möglichst wenige Geldscheine bzw. Münzen** zusammensetzt.

Durch Nachdenken kommen Sie darauf, dass es zur Erreichung des Ziels hilfreich ist, immer möglichst große Geldwerte auszugeben und danach basierend auf der Restsumme mit dem nächstkleineren Geldwert wiederum so zu verfahren.

Hinweis: Die jeweils darzustellende Geldsumme sollten Sie wie gehabt als Konstante an zentraler Stelle in Ihrem Programm ablegen.

Das Ergebnis der Berechnung (also wie viele Scheine bzw. Münzen je Wert benötigt werden) soll dabei **in einem Array** abgelegt werden, das für jede Wertigkeit die entsprechende Anzahl an Scheinen/Münzen enthält.

Wichtig: Verwenden Sie Schleifen und Arrays um Ihren Code effizient zu gestalten.

Testen Sie Ihr Programm, indem Sie die berechneten Werte im Array mit dem Debugger visualisieren.



Schreiben Sie ein Hauptprogramm, welches den Kassen-Algorithmus implementiert.



Testen Sie Ihr Programm mit dem Debugger.

2.3 Verbesserung der Modularität

Sie haben vermutlich schon festgestellt, dass Ihr Hauptprogramm noch etwas unübersichtlich ist.

Um diesem Missstand abzuhelpen, sollte Ihre Funktion aus Abschnitt 2.1, welche die Anzahl an Geldeinheiten berechnet, auch direkt die neue Restsumme berechnen.

Dafür muss die Funktion **zwei** Ergebnisse zurückliefern:

1. Wie bisher die Anzahl der benötigten Geldeinheiten
2. Zusätzlich die Restsumme

Da Funktionen nur **einen** Rückgabewert haben können, benötigen wir hier einen neuen Ansatz²: Strukturdatentypen, die die spezielle Eigenschaft besitzen, dass Variablen dieser Typen auch Werte unterschiedlichen Typs gleichzeitig in sich aufnehmen können.



Verbessern Sie Ihre Funktion zur Stückelungsermittlung

Dozentenfrage

Einsatz der Struktur prüfen und Diskutieren

²Es existieren prinzipiell zwei Lösungen für dieses Problem: Pointer und Strukturdatentypen. Pointer lernen Sie später in einer Folgeaufgabe kennen.

Die Sprache C gestattet es (wie nahezu alle Programmiersprachen) neue Datentypen zu definieren. Bei den sogenannten Strukturdatentypen werden bestehende Datentypen zu neuen Datentypen kombiniert, also gewissermaßen in eine „Kiste“ gepackt, welche man dann als Einheit behandeln kann.

Syntaktisch ist die Definition von Strukturen einfach, wobei Strukturen alle zum Zeitpunkt der Definition des Strukturdatentyps bereits bekannten Datentypen (also auch selbstdefinierte Strukturdatentypen) enthalten dürfen:

```
struct myNiceNewStruct
{
    int FirstElement;
    float SecondElement;
    char AnotherElement;
    double EvenMoreElements[4];
};
```

Ebenso einfach ist die Verwendung der Strukturdatentypen: Variablen können wie gewohnt unter Angabe des Typnamens definiert werden, wobei gewissermaßen der „Familienname“ `struct` stets dem „Vornamen“ (im Beispiel oben `myNiceNewStruct`) voranzustellen ist. Dabei sind natürlich auch Arrays von Strukturen erlaubt.

Für den Zugriff auf die Einzelelemente innerhalb einer auf einer Struktur beruhenden Variable ist ein neuer Operator, der Elementzugriffsoperator, nötig, der oft auch freundschaftlich als „Punkt“-Operator bezeichnet wird.

Hier einige Beispiele:

```
struct myNiceNewStruct myFirstStructVariable=
    {21,23.4, 'P', {1,2,3,4}};
struct myNiceNewStruct mySecondStructVariable;
struct myNiceNewStruct myFirstStructVariableArray[42];

myFirstStructVariable.FirstElement=5
int i;
i = myFirstStructVariable.FirstElement;
myFirstStructVariable.AnotherElement='h';
```

Wichtig: Beachten Sie folgende Schritte bei der Umsetzung gründlich! Gehen Sie bitte so vor, wie beschrieben, Sie riskieren sonst die erfolgreiche Abgabe – oder zeitaufwendige Fehlersuchen.

Gehen Sie nun folgendermaßen vor:

1. **Definieren Sie einen Strukturdatentyp**, welcher es Ihnen gestattet, die Anzahl der Geldeinheiten **und** die neue Restsumme zu speichern.
2. **Ändern Sie Ihre Funktion** aus Teilaufgabe 2.1 so ab, dass Sie als **Rückgabety**p diesen Strukturdatentyp erhält.

3. **Ändern Sie den Programmcode der Funktion** so, dass die neue Restsumme berechnet und zusammen mit der ermittelten Anzahl an Geldeinheiten in einer Variable Ihres neuen Strukturdatentyps abgespeichert und zurückgegeben wird.
4. **Ändern Sie nun Ihr Hauptprogramm** aus Teilaufgabe 2.2 so ab, dass es mit dem neuen Rückgabetyt umgehen kann und die zurückgelieferte neue Restsumme für die weitere Berechnung verwendet.

Wichtig: Testen Sie nun Ihr Programm intensiv, indem Sie verschiedene Summen verarbeiten lassen.

2.4 Programmausgabe

Damit Ihr Programm auch ohne Debugger sinnvoll nutzbar wird, implementieren Sie nun noch **als neue eigenständige Funktion** eine Ausgabe des Ergebnisses in der folgenden Form:

Der Gesamtbetrag von 42 Euro und 46 Cent kann aus

```
0 mal 500-Euro-Schein
0 mal 200-Euro-Schein
0 mal 100-Euro-Schein
0 mal 50-Euro-Schein
2 mal 20-Euro-Schein
0 mal 10-Euro-Schein
0 mal 5-Euro-Schein
1 mal 2-Euro-Münze
0 mal 1-Euro-Münze
0 mal 50-Cent-Münze
2 mal 20-Cent-Münze
0 mal 10-Cent-Münze
1 mal 5-Cent-Münze
0 mal 2-Cent-Münze
1 mal 1-Cent-Münze
```

zusammengesetzt werden.

Die Funktion soll die benötigten Daten für die Ausgabe dabei **per Parameter** übergeben bekommen und nur **einen Aufruf** benötigen, um ein Ergebnis analog zu obigem Beispiel zu erzeugen.

Frage: Überlegen Sie sich vor Beginn der Programmierarbeit, welche Informationen die Funktion benötigt, um ihre Aufgabe erfüllen zu können. Welche Parameter sind daher erforderlich?

Verwenden Sie möglichst Schleifen und vermeiden Sie redundante Codeabschnitte bei der Implementierung.

Hinweis: Testen Sie Ihr Programm gründlich.



Implementieren Sie eine Funktion zur Ausgabe der Berechnungsergebnisse auf dem Bildschirm.

Dozentenfrage

Parameter für die Ausgabefunktion

Frage: Falls Sie die Geldsumme aus dem Beispiel verwenden, wird Ihr Programm wahrscheinlich ein fehlerhaftes Ergebnis produzieren. Überlegen Sie, warum dies so ist und welche Korrekturmaßnahmen man ggf. anwenden könnte.

Korrigieren Sie *nicht* Ihr Programm.

Dozentenfrage

Rundungseffekte bei Fließkommazahlen

Dozententipps

- ? Diskussion der gewählten Lösung:** Hier kann man zum Beispiel eine Integer-Division verwenden, oder eine **float**-Division mit anschließender Abrundung, oder auch eine Schleife mit mehrfacher Subtraktion. Man könnte zum Beispiel diskutieren, welche Vor- und Nachteile die unterschiedlichen Varianten haben. 2
- ? Einsatz der Struktur prüfen und Diskutieren:** Da manchmal der Einsatz des Strukturdatentyps „vergessen“ wird, sollte man den Einsatz der Struktur etwas diskutieren und kann dabei auch häufig einige Missverständnisse rund um die Verwendung der Strukturdatentypen aufklären und das Bild der Kiste für verschiedene Daten bei den Studierenden verankern. 3
- ? Parameter für die Ausgabefunktion:** Hier kann man unterschiedliche Ansätze vertreten. Natürlich wird das Array mit der Stückelung benötigt. Auch werden die Werte der einzelnen Scheine und Münzen benötigt. Und die Anzahl der unterschiedlichen unterstützten Einheitenwerte. Letzteren könnte man als Parameter oder auch als globale Präprozessorkonstante der Funktion zur Verfügung stellen. Mit Literalkonstanten direkt in der Funktion hinterlegte Werte sollte man kritisch hinterfragen wegen der Gefahr von Inkonsistenzen im Programm. 5
- ? Rundungseffekte bei Fließkommazahlen:** Hier sollte man unbedingt diskutieren, warum es zu den beobachteten Abweichungen bei manchen Gesamtsummen kommt. Es sollte bei den Studierenden nicht der Eindruck entstehen „Da nehmen wir einfach **double**, dann wird schon alles gut.“. Vielmehr sollte am Ende die Erkenntnis stehen, dass abzählbare Dinge (wie Geld oder Schrauben) ausschließlich mit Ganzzahl-Datentypen dargestellt werden sollten. Die Summierung von drei mal $1/3$ im Dezimalsystem kann man gut zur Erläuterung verwenden. Das Problem fußt ja im Grunde darin, dass $1/10$ im Binärsystem ebenfalls ein unendlicher Dualbruch ist. . . . 6